Weighted A* Search on the N-Puzzle Problem

Md .Sabbir Ahmmed Payel
2205040
CSE 318 – Offline 1

1 Introduction

The N-puzzle is a classic problem in artificial intelligence, where tiles must be slid into their correct positions using the blank space. The A* search algorithm is widely used to find optimal solutions to such problems. A* evaluates states using the cost function

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost to reach node n from the start, and $h(n)$ is a heuristic estimate of the remaining cost to the goal. The algorithm guarantees an optimal solution when $h(n)$ is admissible (never overestimates) and consistent.

Weighted A*

Weighted A* modifies the evaluation function to

$$f(n) = g(n) + W \cdot h(n)$$

with $W \geq 1.0$. Higher values of W place more emphasis on the heuristic, making the search more greedy and faster, at the cost of optimality. Lower values (close to 1.0) behave like classic A* and produce optimal paths but explore more nodes.

2 Heuristic Design

We combine two heuristics to create an informed and admissible estimate:

- **Manhattan Distance**: For each tile, the sum of the absolute differences between its current and goal coordinates. This is admissible because each tile must move at least that many steps.
- **Linear Conflict**: When two tiles are in their correct row (or column) but in the wrong order relative to each other, at least one must move out and back, adding a penalty of 2 moves per conflict. This improves upon Manhattan distance while remaining admissible.

The combined heuristic is:

$$h(n) = \text{Manhattan}(n) + \text{LinearConflict}(n)$$

This heuristic is both admissible and consistent, ensuring that A* finds the optimal solution when $W = 1.0$.

3 Experimental Setup

We generated 10 distinct solvable 4×4 puzzle configurations. For each configuration, we ran Weighted A* with $W \in \{1.0, 1.2, 2.0, 5.0\}$ and recorded the solution cost (number of moves) and the number of nodes explored.

4 Results

Summary Table

Table 1: Average metrics across 10 test cases for each weight value

Weight	Avg. Nodes Explored	Avg. Cost	Avg. Cost Ratio
1.0	61 344.6	37.8	1.000
1.2	12 379.0	38.6	1.021
2.0	1646.7	46.6	1.233
5.0	422.9	60.6	1.603

Detailed Results

Table 2: Solution cost (number of moves) for each test case across different weights

Test Case	Weight W			
	1.0	1.2	2.0	5.0
0	46	46	60	62
1	40	42	54	72
2	44	44	56	70
3	42	42	50	72
4	28	28	28	56
5	34	34	40	44
6	44	46	52	74
7	42	42	48	60
8	28	30	36	52
9	30	32	42	44
Avg	37.8	38.6	46.6	60.6

5 Analysis and Conclusion

The results clearly demonstrate the trade-off between computational efficiency and solution optimality inherent in Weighted A*:

- $W = 1.0$: Produces the most optimal solutions (lowest cost) in all cases. However, the number of nodes explored is extremely high – sometimes exceeding 200,000

Table 3: Nodes explored for each test case across different weights

Test Case	Weight W			
	1.0	1.2	2.0	5.0
0	205 614	13 113	3679	285
1	5363	2243	828	288
2	32 541	4771	664	154
3	50 985	8587	1196	278
4	707	358	1727	1801
5	15 099	7445	3841	235
6	174 411	54 713	2060	276
7	127 135	31 576	1189	199
8	463	533	220	463
9	1128	651	1063	250
Avg	61 344.6	12 379.0	1646.7	422.9

nodes for a single puzzle. This makes it impractical for time-sensitive applications or larger puzzle sizes.

- $W = 5.0$: Runs extremely fast, exploring only a few hundred nodes on average. However, the solution quality degrades significantly – costs are on average 60% higher than optimal. The greedy nature of the search causes it to make locally optimal choices that lead to suboptimal global paths.

- $W = 2.0$: Offers a reasonable middle ground, with costs only 23% above optimal while reducing explored nodes by over 97% compared to $W = 1.0$.

- $W = 1.2$: Strikes an excellent balance. The average cost is only 2% above optimal, while the number of nodes explored drops by roughly 80% compared to $W = 1.0$. This makes $W = 1.2$ a strong candidate for practical use where near-optimal solutions are acceptable and computation time matters.

In conclusion, while $W = 1.0$ guarantees optimality, the computational cost is prohibitive for complex puzzles. A small weight like $W = 1.2$ preserves near-optimal solution quality while providing substantial speedups, making it the recommended choice for most scenarios.